

FIAP - FACULDADE DE INFORMATICA E ADMINISTRACAO PAULISTA
CIENCIA DA COMPUTACAO

ARTHUR PRIMO BRANDAO - RM 573572
FELIPE GOUVEIA BRAGA - RM 568956
LEANDRO CAVACCINI BRITO - RM 570556
LUCAS DORICE DOS SANTOS - RM 568692
VINICIUS DE OLIVEIRA COPPOLA - RM 571699

CHARGEGRID HUB
SPRINT 3 - PROTOTIPAGEM FUNCIONAL E INTEGRACAO

SAO PAULO
2026

ARTHUR PRIMO BRANDAO; FELIPE GOUVEIA BRAGA; LEANDRO CAVACCINI
BRITO; LUCAS DORICE DOS SANTOS; VINICIUS DE OLIVEIRA COPPOLA

CHARGEGRID HUB

SPRINT 3 - PROTOTIPAGEM FUNCIONAL E INTEGRACAO

Relatorio tecnico-academico apresentado a disciplina de Logica Computacional e Automacao com Python, no curso de Ciencia da Computacao da FIAP, como documentacao da Sprint 3 do projeto desenvolvido em parceria com a GoodWe.

Professor: Alexandre Russi Junior.

SAO PAULO

2026

RESUMO

O ChargeGrid Hub é uma proposta de sistema inteligente para operação comercial de carregadores de veículos elétricos, desenvolvida no contexto de um projeto acadêmico em parceria com a GoodWe. A Sprint 3 tem como objetivo demonstrar a integração funcional dos componentes da solução, reunindo interface de atendimento ao motorista, painel administrativo, motor de simulação, controle dinâmico de demanda, tarifação, comunicação em tempo real, protocolos de recarga e uma ponte para automação física. Nesta etapa, o núcleo do sistema foi centralizado em um servidor Node.js, que mantém uma única instância de simulação e distribui o estado atualizado para as interfaces por WebSocket. O protótipo utiliza dados simulados para representar carga do estabelecimento, geração fotovoltaica, sessões de recarga, potência, estado de carga da bateria, preço e faturamento. O controle de demanda considera uma janela de 15 minutos e ajusta a corrente das estações para evitar ultrapassagem do limite operacional. A solução também demonstra interoperabilidade por Modbus e OCPP 1.6J, além de uma integração MQTT preparada para ESP32 e rele. O fluxo de pagamento Pix está implementado no backend, embora sua aprovação ponta a ponta dependa de credenciais externas válidas. Os resultados indicam que o protótipo é capaz de demonstrar, de forma coerente e rastreável, a cadeia entre entrada de dados, tomada de decisão automática, comando aos componentes e atualização das interfaces.

Palavras-chave: veículos elétricos; controle de demanda; automação; OCPP; Modbus; MQTT; energia solar; tarifação dinâmica.

SUMARIO

1 INTRODUCAO	1
1.1 Contextualizacao e problema	1
1.2 Objetivos da Sprint 3	2
2 METODOLOGIA E ESCOPO DO PROTOTIPO	3
3 ARQUITETURA E INTEGRACAO DOS COMPONENTES	4
3.1 Arquitetura centralizada e tempo real	4
3.2 Jornada funcional de uma sessao	6
4 CONTROLE DINAMICO DE DEMANDA	7
5 SUSTENTABILIDADE E TARIFACAO DINAMICA	9
6 PROTOCOLOS, AUTOMACAO E HARDWARE	11
7 BACKEND, SEGURANCA E PAGAMENTO PIX	13
8 RESULTADOS E DADOS FUNCIONAIS	15
9 CONEXAO COM OS CONTEUDOS DA DISCIPLINA	17
10 ORGANIZACAO DO REPOSITORIO E EXECUCAO	18
11 ADERENCIA AOS CRITERIOS DE AVALIACAO	19
12 CONCLUSAO	20
REFERENCIAS	21
APENDICE A - MODELO DO ARQUIVO TXT	22

1 INTRODUCAO

1.1 Contextualizacao e problema

A expansao da mobilidade eletrica cria uma demanda crescente por pontos de recarga fora do ambiente residencial. Em estabelecimentos comerciais, empresas, estacionamentos e condominios, a instalacao de varios carregadores introduz problemas que nao aparecem com a mesma intensidade em uma instalacao domestica: a potencia disponivel precisa ser compartilhada, a demanda contratada do local deve ser respeitada, o motorista precisa compreender quanto esta pagando e o operador necessita visualizar o desempenho energetico e financeiro do conjunto.

O ChargeGrid Hub foi concebido para atuar sobre esse problema. A proposta nao se limita a apresentar uma interface para um carregador; o sistema transforma o conjunto de pontos de recarga em uma operacao coordenada. O totem organiza a jornada do motorista, o motor de gerenciamento calcula a potencia e o preco de cada sessao, e o painel administrativo converte o estado tecnico do sistema em informacoes de operacao. Esses componentes sao integrados por um servidor central, responsavel por manter a mesma fonte de verdade para todas as telas e protocolos.

Na Sprint 3, a principal exigencia e demonstrar um prototipo funcional, ainda que parte dos dados e equipamentos seja simulada. Por esse motivo, o foco deste relatorio esta na integracao: como uma entrada de usuario gera uma sessao, como essa sessao recebe energia de acordo com a disponibilidade do estabelecimento, como os comandos sao representados em protocolos tecnicos e como as alteracoes retornam para o usuario e para o operador em tempo real.

1.2 Objetivos da Sprint 3

O objetivo geral desta etapa e avancar na implementacao tecnica do ChargeGrid Hub e comprovar que os principais componentes da solucao podem atuar de forma integrada em um prototipo funcional. A demonstracao considera tanto elementos efetivamente executados em software quanto elementos simulados de maneira controlada, preservando a distincao entre o que ja foi validado e o que ainda depende de hardware ou credenciais externas.

Como objetivos especificos, a Sprint 3 procura:

- Demonstrar a comunicacao entre totem, servidor, motor de simulacao e painel administrativo.
- Justificar as tecnologias adotadas para tempo real, protocolos, automacao, pagamento e simulacao energetica.
- Apresentar dados funcionais de sessoes de recarga, potencia, corrente, energia, custo, demanda, geracao solar e faturamento.
- Evidenciar o funcionamento do controle automatico de demanda e da tarifacao dinamica em diferentes cenarios.
- Documentar o codigo, os scripts, o firmware e as instrucoes de funcionamento no repositorio do projeto.

2 METODOLOGIA E ESCOPO DO PROTOTIPO

O desenvolvimento da Sprint 3 adotou uma abordagem de prototipagem funcional orientada por simulacao. Em vez de depender de quatro carregadores fisicos, multiplos veiculos e uma instalacao eletrica de grande porte, o sistema utiliza modelos computacionais para representar o comportamento do estabelecimento, das sessoes de recarga e dos equipamentos. Essa estrategia permite testar a logica de integracao em um ambiente controlado e reproduzivel, sem afirmar que os valores simulados sejam leituras de um equipamento fisico real.

O prototipo considera quatro pontos de recarga baseados no perfil do carregador GoodWe HCA G2, cada um com potencia nominal simulada de 7,36 kW e corrente maxima de 32 A. O estabelecimento e modelado com demanda contratada de 75 kW e margem operacional de seguranca de 5%, resultando em limite de 71,25 kW para a logica de controle. A geracao fotovoltaica simulada tem potencia instalada de 90 kWp, com uma curva diaria que permite representar periodos de maior disponibilidade solar.

A avaliacao funcional foi estruturada por cenarios: operacao normal com folga de potencia, aumento moderado da carga do predio, pico severo que exige reducao ou suspensao de recarga, mudanca de horario tarifario, excedente solar, conclusao de sessao, comunicacao MQTT e tentativa de integracao Pix. Em cada caso, a evidencia esperada e o efeito produzido no estado do sistema, nas interfaces e nos registros de protocolo.

Tabela 1 - Escopo funcional da Sprint 3

Componente	Funcao no prototipo	Situacao
Totem	Selecao de estacao e veiculo, estimativa, pagamento, acompanhamento e encerramento da recarga.	Funcional em software
Motor de simulacao	Calcula carga, geracao solar, energia, SoC, preco, demanda e historico.	Funcional em software
Painel administrativo	Exibe KPIs, estacoes, sessoes, energia, tarifacao e registros tecnicos.	Funcional em software
Modbus e OCPP	Representam comandos e eventos associados ao controle dos carregadores.	Simulados e demonstraveis
MQTT e ESP32	Ponte para acionamento fisico por rele e leitura de sensor.	Firmware pronto; montagem fisica pendente
Pix	Criacao e consulta de cobranca no backend.	Implementado; aprovacao depende de credencial valida

Fonte: Elaborado pelos autores com base no repositório do projeto (2026).

3 ARQUITETURA E INTEGRACAO DOS COMPONENTES

3.1 Arquitetura centralizada e comunicacao em tempo real

A decisao arquitetural mais relevante da Sprint 3 foi centralizar o estado da simulacao no servidor. Em uma versao anterior ou em uma implementacao exclusivamente visual, cada pagina poderia manter seus proprios dados, o que criaria divergencia entre o que o motorista observa no totem e o que o operador observa no painel administrativo. A arquitetura atual evita esse problema ao manter uma instancia persistente do motor de simulacao no backend.

O servidor Node.js, apoiado pelo framework Express, concentra APIs de apoio, integracao com pagamento e a instancia do SimEngine. O estado produzido pelo motor e publicado aos clientes por meio de WebSocket. Dessa forma, o totem e o painel administrativo recebem periodicamente o mesmo snapshot, incluindo sessoes ativas, status de cada EVSE, potencia, corrente, SoC, demanda do estabelecimento, geracao solar, preco e indicadores acumulados.

O caminho inverso tambem e integrado. Comandos originados no totem ou no painel chegam ao servidor pelo WebSocket e sao processados pelo motor. Uma alteracao de corrente determinada pelo controlador pode resultar simultaneamente em atualizacao do estado da sessao, geracao de escrita Modbus, emissao de SetChargingProfile na camada OCPP e atualizacao das duas interfaces. Essa cadeia e uma das principais evidencias de funcionamento integrado da Sprint.

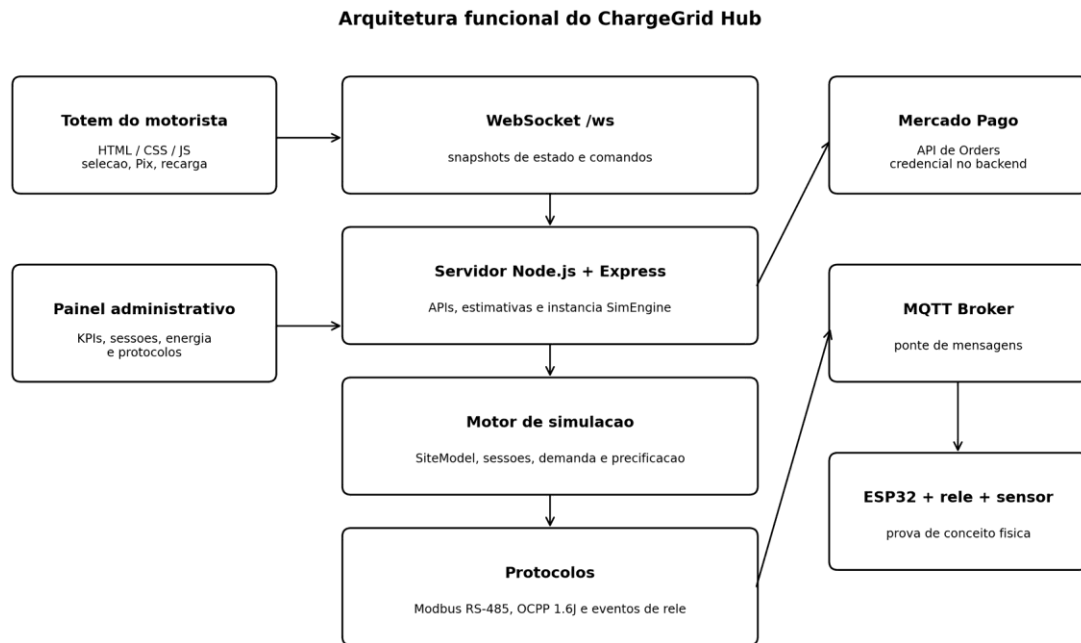


Figura 1 - Arquitetura funcional do ChargeGrid Hub

Fonte: Elaborado pelos autores com base na estrutura do repositório (2026).

A arquitetura também foi escolhida para facilitar a extensão do projeto. A interface do motorista permanece desacoplada das credenciais de pagamento, o painel administrativo não precisa conhecer os detalhes internos de cada protocolo e o motor pode ser evoluído sem alterar a forma como os clientes recebem o estado. A separação por responsabilidades reduz o acoplamento entre os módulos e torna o protótipo mais próximo de uma aplicação de engenharia de software do que de uma demonstração exclusivamente visual.

3.2 Jornada funcional de uma sessão

A jornada funcional inicia quando o motorista seleciona uma estação disponível no totem. Em seguida, o sistema recebe os dados do veículo, como capacidade de bateria, limite de carga e estado inicial de carga. Com essas informações, o backend calcula uma estimativa de energia, tempo e custo. O fluxo pode seguir para pagamento Pix ou, em ambiente de demonstração, utilizar um modo de simulação que permite testar a recarga mesmo sem uma transação aprovada.

Ao iniciar a recarga, o servidor cria a sessão e registra os eventos correspondentes na camada OCPP. O motor passa a atualizar a sessão a cada ciclo de simulação, calculando energia entregue, variação de SoC, potência instantânea, custo acumulado e tempo restante. Em paralelo,

o controlador de demanda avalia a disponibilidade energetica do local e define a corrente maxima autorizada para cada EVSE.

Do ponto de vista do motorista, essas decisoes tecnicas sao convertidas em informacoes simples. A tela mostra percentual de bateria, potencia, corrente autorizada, energia fornecida, tempo decorrido, custo e preco atual. Quando a demanda do estabelecimento se aproxima do limite, o usuario recebe um aviso de reducao de potencia. O painel administrativo recebe o mesmo evento e passa a exibir o ponto como limitado ou suspenso, preservando a consistencia entre as duas perspectivas.

Quando a sessao e encerrada, o motor consolida os dados realmente produzidos naquela execucao. O historico pode armazenar energia, custo e segmentos tarifarios atravessados durante a recarga, o que permite explicar por que o valor final pode ter sido composto por diferentes precos ao longo do tempo.

4 CONTROLE DINAMICO DE DEMANDA

O controle dinamico de demanda e o principal mecanismo de eficiencia energetica do ChargeGrid Hub. Seu objetivo e permitir a recarga de veiculos sem ultrapassar a capacidade operacional do estabelecimento. Em vez de conceder sempre a corrente maxima a todos os carregadores, o sistema calcula quanta potencia pode ser usada naquele momento e distribui essa disponibilidade entre as sessoes ativas.

4.1 Premissas eletricas e janela de medicao

O modelo utiliza uma demanda contratada de 75 kW. Para incorporar uma margem operacional, o sistema considera 95% desse valor como limite de trabalho, equivalente a 71,25 kW. A potencia disponivel para os veiculos, denominada headroom, depende da carga do predio e da geracao solar. Quando a geracao renovavel reduz a demanda liquida, mais potencia pode ser liberada para recarga; quando a carga do predio aumenta, o controlador reduz a potencia dos EVSEs.

A decisao nao depende somente da leitura instantanea. O controlador acumula energia em uma janela de 900 segundos, ou 15 minutos, e utiliza essa informacao para projetar a media de fechamento da demanda. Essa abordagem e importante porque a contratacao de energia e os

limites operacionais frequentemente são avaliados em intervalos e não apenas por picos instantâneos de curta duração.

4.2 Estados de operação e distribuição de corrente

O comportamento do controlador é organizado em quatro estados: NORMAL, ATENÇÃO, LIMITANDO e SUSPENSO. Em condição normal, os carregadores podem operar próximo de seus limites individuais. Quando a margem diminui, o sistema entra em atenção e passa a preparar a redução. No estado limitando, a corrente autorizada é reduzida. Se a potência disponível cair a ponto de não permitir a corrente mínima configurada, uma sessão pode ser temporariamente suspensa e a potência restante é redistribuída.

O protótipo utiliza corrente máxima de 32 A por EVSE e corrente mínima configurada de 6 A. A distribuição procura respeitar prioridade e disponibilidade, utilizando uma lógica de alocação do tipo water-fill. Na prática, isso significa que o sistema tenta compartilhar a potência entre as sessões antes de suspender um usuário; a suspensão ocorre quando a divisão disponível deixa de ser suficiente para manter uma recarga mínima estável.

4.3 Protocolos de estabilidade

Para evitar oscilações excessivas de comando, o controlador incorpora mecanismos de estabilidade. O degrau mínimo de alteração é de 2 A, o intervalo mínimo entre comandos é de 20 segundos simulados e o tempo de espera para retomada após uma suspensão é de 60 segundos simulados. Esses parâmetros funcionam como histerese operacional e impedem que pequenas variações da carga provoquem alterações contínuas e desnecessárias na corrente dos carregadores.

Tabela 2 - Parâmetros principais do controle de demanda

Parâmetro	Valor utilizado no protótipo
Demanda contratada	75 kW
Limite operacional	71,25 kW
Janela de demanda	15 minutos (900 s)
Corrente máxima por EVSE	32 A
Corrente mínima configurada	6 A
Degrau mínimo de ajuste	2 A
Intervalo mínimo entre comandos	20 s simulados
Hold de retomada	60 s simulados

Fonte: Elaborado pelos autores com base em server/sim/demand.js, server/sim/site.js e DEMO.md (2026).

5 SUSTENTABILIDADE E TARIFACAO DINAMICA

5.1 Integracao da geracao solar

A sustentabilidade é incorporada ao prototipo por meio de uma curva de geração fotovoltaica simulada. A configuração representa um sistema de 90 kWp e permite observar o efeito da produção solar ao longo do dia. Em horários de maior irradiação, especialmente entre o fim da manhã e o início da tarde, a geração reduz a demanda líquida do estabelecimento e pode criar excedente energético disponível para a recarga dos veículos.

No ChargeGrid Hub, a energia solar não é apenas um indicador visual. Sua contribuição altera o headroom calculado pelo controle de demanda e também participa da formação do preço. Essa integração transforma a disponibilidade renovável em uma variável operacional: quando existe mais energia solar, o sistema pode liberar mais potência para os EVs e, simultaneamente, reduzir o custo relativo da energia usada na sessão.

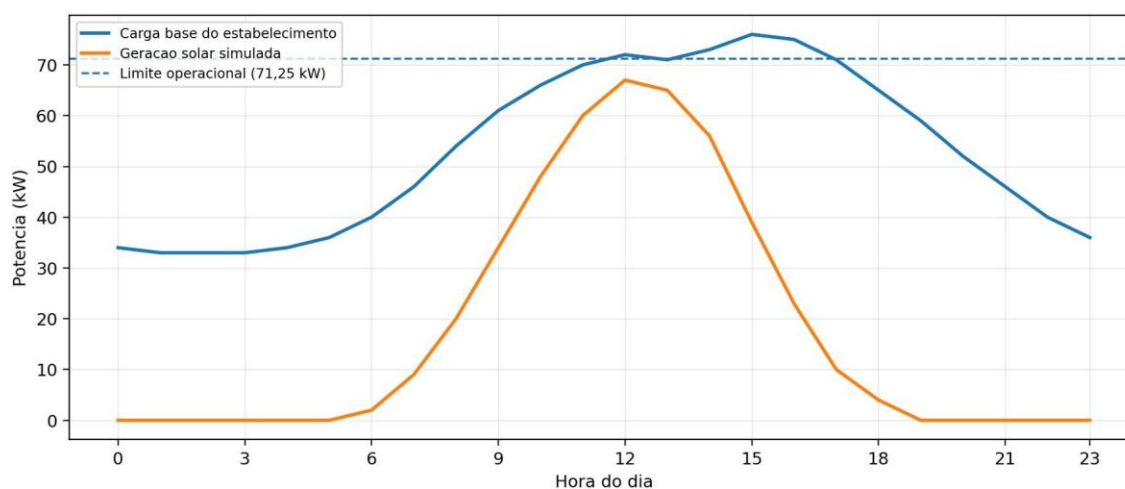


Figura 2 - Curvas representativas de carga, geração solar e limite operacional

Fonte: Elaborado pelos autores a partir da configuração de simulação do projeto (2026).

5.2 Smart Pricing

A tarifação dinâmica procura sinalizar ao usuário o custo operacional de diferentes horários e condições. O preço final considera componentes do custo de rede, como TE, TUSD, bandeira e tributos, e adiciona margem comercial. Em seguida, fatores dinâmicos modificam o valor de acordo com horário, ocupação do eletroposto, disponibilidade solar e escassez de potência.

O prototipo aplica piso e teto de preco, com valores de R\$ 0,55/kWh e R\$ 3,20/kWh. O guia de demonstracao registra tres cenarios de referencia: aproximadamente R\$ 0,96/kWh ao meio-dia, com excedente solar; cerca de R\$ 1,53/kWh as 15h, em operacao intermediaria; e R\$ 3,08/kWh as 19h, em horario de ponta e com ocupacao elevada. Esses valores sao cenarios de simulacao, e nao tarifas comerciais reais da GoodWe ou de uma concessionaria especifica.

A combinacao entre controle de demanda e precificacao permite duas formas de gestao. A primeira e fisica: limitar potencia para respeitar a capacidade eletrica. A segunda e economica: comunicar ao motorista que determinados horarios apresentam menor ou maior custo. Em uma implementacao comercial futura, esse sinal pode contribuir para deslocar parte da demanda para periodos com maior disponibilidade energetica.

6 PROTOCOLOS, AUTOMACAO E HARDWARE

A Sprint 3 incorpora camadas de protocolo para demonstrar que as decisoes do software podem ser traduzidas em comandos tecnicos. Essa caracteristica e importante porque um sistema de recarga comercial precisa interoperar com equipamentos fisicos e plataformas de gestao. No prototipo, Modbus, OCPP e MQTT cumprem funcoes diferentes e complementares.

6.1 Modbus RS-485

A camada Modbus representa a comunicacao de baixo nivel com os carregadores. Os quatro pontos simulados utilizam slave IDs de 11 a 14. Quando o controlador altera a corrente maxima autorizada, o motor registra uma escrita no registrador correspondente. O guia de demonstracao destaca o registrador 0x0010 e quadros de funcao FC06. O CRC16 e calculado pelo proprio motor de protocolo, permitindo exibir no painel um quadro coerente com uma comunicacao Modbus real.

6.2 OCPP 1.6J

O OCPP e usado para representar eventos de gerenciamento de recarga em um nivel mais alto. A simulacao inclui mensagens como BootNotification e StatusNotification para estado do ponto, Authorize para autorizacao, StartTransaction e StopTransaction para ciclo da sessao, MeterValues durante a recarga e SetChargingProfile quando a corrente e modificada pelo

controle de demanda. Essa camada ajuda a demonstrar que o gerenciamento energetico pode ser refletido em comandos compatíveis com uma arquitetura de eletroposto conectado.

6.3 MQTT, ESP32 e rele

A ponte de automação física utiliza MQTT para transportar comandos simples de LIGAR e DESLIGAR entre o software e um ESP32. O firmware presente no repositório contempla conexão Wi-Fi, cliente MQTT, controle de rele e leitura de sensor. A ideia é que o evento de início ou fim de uma sessão possa gerar uma ação física observável em uma bancada de demonstração.

O repositório registra que o round-trip MQTT foi validado por emulação: o navegador publicou o comando, um processo representando o ESP32 recebeu a mensagem e devolveu o status. A montagem com ESP32 real, rele e sensor permanece dependente do hardware do grupo. Por isso, o relatório classifica o firmware e a lógica de comunicação como implementados, mas não apresenta a montagem física como validação concluída.

Tabela 3 - Situação das camadas de protocolo e automação

Camada	Evidência	Status
Modbus	Escrita de corrente, quadro FC06 e CRC16 no motor.	Demonstrável em simulação
OCPP 1.6J	Eventos de autorização, transação, telemetria e perfil de carga.	Demonstrável em simulação
MQTT	Publicação LIGAR/DESLIGAR e retorno de status.	Round-trip validado por emulação
ESP32/rele/sensor	Firmware esp32/chargegrid-rele.ino.	Montagem física pendente

Fonte: Elaborado pelos autores com base em DEMO.md, HARDWARE-E-PAGAMENTO.md e firmware do repositório (2026).

7 BACKEND, SEGURANÇA E PAGAMENTO PIX

7.1 Backend e separação de responsabilidades

A integração com pagamento foi colocada no backend para impedir que credenciais sensíveis sejam expostas no navegador. O Access Token do provedor de pagamento é lido por variável de ambiente no servidor, enquanto o totem utiliza apenas rotas locais da aplicação. Essa separação segue uma prática essencial de segurança: segredos de integração não devem ser incorporados diretamente ao código entregue ao cliente.

Entre as rotas implementadas estão a criação de cobrança Pix, a consulta de status, a verificação de saúde do serviço e a estimativa da sessão. A implementação também utiliza

validacao de valor, chave de idempotencia para reduzir duplicidade de cobranças e tratamento explicito de falhas. O objetivo da Sprint não é certificar uma plataforma financeira, mas demonstrar que o fluxo comercial pode ser integrado ao mesmo backend que coordena a recarga.

7.2 Fluxo de pagamento e estado de validacao

No fluxo previsto, o totem envia o valor e a descricao da cobrança ao servidor. O backend cria uma Order na API do Mercado Pago e devolve os dados de QR Code e código copia-e-cola. O totem passa a consultar o estado da cobrança e somente após a aprovacao o fluxo segue para a recarga. A interface inclui estados de carregamento, exibicao de valor, código de pagamento e tratamento de erro.

A comunicacao externa foi parcialmente validada: a documentacao registra que a requisicao chegou ao Mercado Pago, mas recebeu recusa devido a um token de teste invalido. Isso comprova o caminho de integracao e o tratamento de resposta, mas não comprova uma aprovacao ponta a ponta. Para manter a documentacao tecnicamente correta, a Sprint diferencia a existencia do código de pagamento da validacao completa da transacao.

8 RESULTADOS E DADOS FUNCIONAIS

Os resultados da Sprint 3 são apresentados como evidencias funcionais do comportamento do prototipo. Os valores de energia, preço, demanda e sessoes são produzidos pelo motor de simulacao durante a execucao. Portanto, devem ser interpretados como dados funcionais simulados, e não como medicoes obtidas diretamente de um carregador HCA G2 fisico.

Tabela 4 - Cenarios de teste e resultados observaveis

Cenario	Resultado observavel	Evidencia tecnica
Sincronismo totem-admin	Sessao iniciada no totem aparece no painel com o mesmo estado.	SimEngine unico e broadcast por WebSocket.
Pico moderado	Corrente é reduzida e a sessao passa ao estado limitado.	DemandController, Modbus e OCPP SetChargingProfile.
Pico severo	Sessao pode ser suspensa quando a alocao fica abaixo de 6 A.	Water-fill, prioridade e corrente minima.
Mudanca de horario	Preço varia entre fora-ponta, intermediario e ponta.	TariffEngine e SmartPricing.
Excedente solar	Solar reduz demanda liquida e influencia o preço.	SiteModel.solarFraction() e fator solar.
Sessao concluida	Energia, custo e segmentos de preço entram no historico.	SimEngine.today, sessionHistory e rateSegments.
MQTT	Comandos LIGAR/DESLIGAR recebem retorno de status.	Validacao por processo emulando o ESP32.
Pix	Backend alcanca API e trata falha de credencial.	Rotas implementadas em server/server.js.

Fonte: Elaborado pelos autores com base em DEMO.md, server/sim/* e HARDWARE-E-PAGAMENTO.md (2026).

8.1 Indicadores e cenários numéricos

A simulação trabalha com quatro pontos de recarga, corrente máxima de 32 A por EVSE, corrente mínima de 6 A e janela de demanda de 15 minutos. No guia de demonstração, a tarifação utiliza três cenários numéricos representativos: R\$ 0,96/kWh ao meio-dia com excedente solar; aproximadamente R\$ 1,53/kWh as 15h em condição intermediária; e R\$ 3,08/kWh as 19h em horário de ponta e alta ocupação. Esses números permitem demonstrar a resposta da precificação dinâmica sem confundir o protótipo com uma tabela tarifária oficial.

A principal evidência de integração é a relação causal entre os dados. Um aumento na carga do estabelecimento reduz o headroom; o controlador diminui a corrente de uma ou mais sessões; a potência entregue aos veículos cai; um comando Modbus e um perfil OCPP são registrados; o status do EVSE muda; e o novo estado é transmitido ao totem e ao painel. O resultado não é apenas uma mudança de interface, mas uma cadeia de decisão rastreável dentro do sistema.

8.2 Limitações da demonstração

A documentação da Sprint deve diferenciar claramente simulação, emulação e validação física. O núcleo de software, o controle de demanda, o sincronismo entre telas e as camadas simuladas de protocolo podem ser demonstrados integralmente no ambiente de execução. O round-trip MQTT possui validação por emulação. Já a montagem física do ESP32 e a aprovação completa de uma cobrança Pix dependem, respectivamente, de hardware montado e de credencial externa válida.

Essa classificação não reduz o valor técnico do protótipo; ao contrário, aumenta a rastreabilidade da entrega. Ao registrar exatamente o que foi testado, o grupo evita apresentar como concluída uma etapa que depende de recursos externos e permite que a banca avalie de forma objetiva a arquitetura, o código e os resultados que estão disponíveis.

9 CONEXÃO COM OS CONTEÚDOS DA DISCIPLINA

O ChargeGrid Hub materializa conceitos de lógica computacional e automação em um problema de engenharia aplicada. A lógica de decisão utiliza condicionais para definir os estados

de operacao, estruturas de dados para representar estacoes e sessoes, repeticoes para atualizar a simulacao e funcoes para calculos de tarifa, eficiencia, estado de carga e distribuicao de potencia.

A automacao aparece quando o sistema recebe dados de demanda e, sem intervencao manual, altera a corrente autorizada para cada ponto. O resultado da decisao e convertido em eventos Modbus, OCPP e MQTT. Assim, a programacao deixa de ser apenas processamento numerico e passa a controlar um processo, que e uma caracteristica central de sistemas automatizados.

Os conteudos de dados e simulacao sao observados nas curvas horarias de carga e geracao solar, nos calculos de energia e custo e na producao de indicadores operacionais. A integracao de sistemas e demonstrada por APIs HTTP, WebSocket e protocolos de comunicacao. Embora o motor principal esteja implementado em JavaScript/Node.js, os fundamentos de decomposicao de problema, logica, automacao, estruturas de controle e tratamento de dados sao transferiveis para Python e para outras linguagens estudadas no curso.

O aprendizado mais relevante da Sprint 3 consiste em conectar as camadas. Uma interface isolada pode aparentar funcionamento sem representar o comportamento do sistema. No prototipo atual, a interface recebe resultados de uma logica comum, a logica gera comandos tecnicos e os comandos retornam como estado observavel. Essa continuidade entre entrada, processamento, atuacao e monitoramento e o principal elo com a disciplina.

10 ORGANIZACAO DO REPOSITORIO E EXECUCAO

O repositorio do ChargeGrid Hub foi organizado por responsabilidades. A pasta css contem estilos das interfaces; data armazena configuracoes de veiculos, site e mapa Modbus; esp32 contem o firmware de prova de conceito; js concentra codigo de interface e clientes de simulacao; e server contem o backend, o WebSocket e o motor central. Na raiz estao as paginas principais e documentos de apoio, como README.md, DEMO.md e HARDWARE-E-PAGAMENTO.md.

Para executar o prototipo, o backend deve ser iniciado a partir da pasta server, com instalacao das dependencias e configuracao do arquivo .env. Em seguida, os arquivos estaticos podem ser servidos localmente. O totem e o painel administrativo acessam o mesmo backend, que por padrao utiliza a porta 3001, enquanto a interface estatica pode ser disponibilizada na porta 8322.

Procedimento	resumido	de	execucao:
1.	cd		server
2.	npm		install
3.	configurar		server/.env
4.	node		server.js
Em	outro		terminal:
5.	python	-m	8322
		http.server	
Totem:			http://localhost:8322
Admin:			http://localhost:8322/admin.html
Backend:	http://localhost:3001		

A documentacao e parte do proprio entregavel tecnico. O README apresenta a visao geral e as instrucoes; o DEMO.md registra os cenarios e o roteiro de demonstracao; e o documento HARDWARE-E-PAGAMENTO.md detalha a ponte com ESP32, MQTT e Mercado Pago. Essa separacao permite que a banca navegue tanto pelo produto quanto pelas evidencias de implementacao.

11 ADERENCIA AOS CRITERIOS DE AVALIACAO

A documentacao foi organizada para responder diretamente aos tres criterios definidos para a Sprint 3. O criterio de integracao e funcionamento tecnico e atendido pela demonstracao do motor central, comunicacao em tempo real, controle de demanda, protocolos e interfaces. O criterio de justificativa e alinhamento com a disciplina e atendido pelas decisoes arquiteturais, pelos algoritmos de controle e pela relacao com automacao e tratamento de dados. O criterio de organizacao do repositorio e documentacao e atendido pela separacao do codigo, pelos guias de execucao, pelo firmware e por este relatorio.

Tabela 5 - Relacao entre criterios e evidencias do projeto

Criterio	Pontuacao prevista	Principais evidencias
Integracao e funcionamento tecnico	Ate 50 pontos	Motor unico, WebSocket, controle de demanda, Modbus, OCPP, MQTT, totem e admin.
Justificativa e alinhamento com a disciplina	Ate 25 pontos	Decisoes de engenharia, algoritmos, automacao, simulacao, dados e integracao.
Organizacao do repositorio e documentacao tecnica	Ate 25 pontos	Codigo por camadas, README, DEMO, documentacao de hardware/pagamento e instrucoes de execucao.

Fonte: Elaborado pelos autores a partir do enunciado da Sprint 3 (2026).

12 CONCLUSÃO

A Sprint 3 demonstra a evolução do ChargeGrid Hub de uma proposta de interface para um prototipo integrado. O sistema possui um núcleo central que coordena o estado das sessões e conecta experiência do motorista, operação administrativa, controle energético, protocolos e serviços externos. Essa arquitetura permite que uma mudança de demanda produza efeitos consistentes em toda a cadeia: corrente autorizada, potência, status, registros de protocolo e informações exibidas nas telas.

O controle dinâmico de demanda se destaca como principal mecanismo técnico, pois relaciona diretamente o problema de expansão de carregadores em estabelecimentos com a necessidade de respeitar a infraestrutura elétrica existente. A geração solar e a tarifação dinâmica complementam essa lógica ao incorporar sustentabilidade e sinal econômico. Modbus, OCPP e MQTT aproximam a simulação do contexto de uma instalação conectada e mostram como os comandos de software poderiam chegar aos equipamentos.

Os resultados apresentados são suficientes para demonstrar a integração funcional do núcleo de software e os cenários previstos para a disciplina. Ao mesmo tempo, o projeto mantém transparência sobre os limites da validação: os dados energéticos são simulados, o ESP32 físico ainda depende da montagem e a aprovação ponta a ponta do Pix depende de credencial externa válida. Essa distinção permite avaliar o prototipo pelo que ele efetivamente implementa e deixa claros os próximos passos para uma evolução em ambiente real.

Como continuidade, recomenda-se conectar o sistema a um conjunto físico de sensores e atuadores, validar o comportamento com carregadores reais, registrar dados persistentes em banco de dados e executar testes de estresse com múltiplas sessões. Essas etapas permitiriam comparar o modelo de simulação com medições reais e preparar o ChargeGrid Hub para uma prova de conceito mais próxima de um eletroposto comercial.

REFERÊNCIAS

BRANDAO, Arthur Primo et al. ChargeGrid Hub. GitHub, 2026. Disponível em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. README.md: funcionalidades, instruções de execução e equipe. In: BRANDAO, Arthur Primo et al. ChargeGrid Hub. GitHub, 2026. Disponível em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. DEMO.md: cenarios de demonstracao, precos de referencia, controle de demanda e limitacoes declaradas. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. HARDWARE-E-PAGAMENTO.md: ESP32, rele, sensor, MQTT, Mercado Pago e estado de validacao. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. server/server.js: API Pix, estimativas e inicializacao do motor. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. server/ws.js: sincronizacao em tempo real entre servidor, totem e painel. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. server/sim/engine.js: orquestracao da simulacao, sessoes, OCPP, Modbus e historico. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. server/sim/demand.js: janela de demanda, alocao de potencia, estados e histerese. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. server/sim/site.js: calculo de carga liquida, solar, ocupacao e limite operacional. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. server/sim/smart-pricing.js: preco dinamico por ocupacao, horario, solar e restricao de rede. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. js/sim/tariff.js: postos tarifarios e decomposicao de custos utilizada na simulacao. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. data/site-config.js: demanda contratada, margem, geracao solar e configuracao dos EVSEs. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

CHARGEGRID HUB. esp32/chargegrid-rele.ino: firmware da prova de conceito fisica. GitHub, 2026. Disponivel em: <<https://github.com/ArthurPB25/charge-grid-hub>>. Acesso em: 21 set. 2026.

APENDICE A - MODELO DO ARQUIVO TXT PARA ENTREGA

Conforme as condições da atividade, a submissão na plataforma deve conter apenas um arquivo .txt com nome e RM dos integrantes, link do vídeo no YouTube configurado como não listado e link do repositório GitHub. O conteúdo abaixo pode ser utilizado como modelo, substituindo o campo do vídeo pelo link definitivo.

Arthur	Primo	Brandao	-	RM	573572
Felipe	Gouveia	Braga	-	RM	568956
Leandro	Cavaccini	Brito	-	RM	570556
Lucas	Dorice	Dos Santos	-	RM	568692
Vinicius	de Oliveira	Coppola	-	RM	571699

Video (YouTube não listado): <https://youtu.be/rEnDvuQL-GA>
Repositorio: <https://github.com/ArthurPB25/charge-grid-hub.git>